AVALIAÇÃO		QUESTIONÁRIO: ☐	FICHA: ☒	TRABALHO: ☐	FORMATIVO: ☒	SUMATIVO: ☐
Curso:	EFA Tecnológico – Programador/a de Informática	Turma:	01/25	Data:	20-01-2026	Nº Pág.: 1
UFCD:	10794 – Programação avançada em Python	Formador:	Marcos de Lima Alvarães			Classificação:

Atividade

Formando: Rodrigo Moraes Fontes, 14

RELATÓRIO DO PROJETO CODEVET

Este relatório apresenta o projeto **CodeVet**, um sistema de gestão para clínicas veterinárias desenvolvido em Python com o framework Flask. O objetivo do projeto foi aplicar, na prática, os conhecimentos adquiridos ao longo da formação, nomeadamente na criação de aplicações web, ligação a bases de dados e controlo de acessos por tipo de utilizador. O sistema permite a gestão de clientes, animais e consultas, bem como uma área reservada para clientes da clínica. Ao longo do relatório são apresentados exemplos de código, acompanhados de capturas de ecrã e uma breve explicação das principais funcionalidades implementadas, de forma a demonstrar o funcionamento e a organização do projeto.

PESSOAS 2030 · PORTUGAL 2030 · Cofinanciado pela União Europeia
Os Fundos Europeus mais próximos de si.

De início, usei o ficheiro sql modelo do formador para gerar a minha base de dados. Porém fiz algumas alterações as quais achei mais pertinente para o meu projeto.

- Na tabela clientes, incluí um campo "password" para melhor gerenciamento das senhas;

- Retirei a role cliente_id da tabela users. Dessa forma, tudo que envolve os clientes é gerido separadamente na tabela clientes, garantindo uma divisão mais clara de quem é funcionário da clínica e quem é cliente.

```sql
DROP TABLE IF EXISTS `clientes`;
/*!40101 SET @saved_cs_client     = @@character_set_client */;
/*!50503 SET character_set_client = utf8mb4 */;
CREATE TABLE `clientes` (
  `id` int NOT NULL AUTO_INCREMENT,
  `nome` varchar(120) NOT NULL,
  `telefone` varchar(30) DEFAULT NULL,
  `email` varchar(120) DEFAULT NULL,
  `morada` varchar(255) DEFAULT NULL,
  `created_at` timestamp NULL DEFAULT CURRENT_TIMESTAMP,
  `password` varchar(255) DEFAULT '1234',
  PRIMARY KEY (`id`),
  UNIQUE KEY `email` (`email`)
```

Tabela clientes com o campo password

```sql
DROP TABLE IF EXISTS `users`;
/*!40101 SET @saved_cs_client     = @@character_set_client */;
/*!50503 SET character_set_client = utf8mb4 */;
CREATE TABLE `users` (
  `id` int NOT NULL AUTO_INCREMENT,
  `username` varchar(50) NOT NULL,
  `password` varchar(255) NOT NULL,
  `role` enum('admin','staff') NOT NULL,
  `created_at` timestamp NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `username` (`username`)
```

Tabela users sem o campo cliente_id e roles apenas admin e staff

Feito isto, comecei a criar as rotas e métodos essenciais parao funcionamento da página. O projeto CodeVet possui diversas rotas organizadas por áreas funcionais. Abaixo são apresentadas as **rotas mais relevantes**, que representam a **estrutura, a lógica de negócio e o controlo de permissões** do sistema.

```python
# --------------- PERMISSOES (SEM WRAPS / SEM DECORATORS) ---------------

# Verifica se existe um utilizador autenticado na sessão.
# A sessão é considerada válida quando a chave "user_id" existe,
# o que significa que o login foi efetuado com sucesso anteriormente.
# Retorna True se o utilizador estiver logado, ou False caso contrário.
def esta_logado():
    return "user_id" in session


# Obtém o papel (role) do utilizador atualmente autenticado.
# O role é definido no momento do login (admin ou staff)
# e é utilizado para controlar permissões de acesso às rotas.
# Retorna None caso não exista role na sessão.
def role_atual():
    return session.get("role")


# Retorna o identificador do cliente autenticado.
# Esta informação só existe quando o login é feito como cliente.
# É utilizada para filtrar dados específicos do cliente,
# como animais, consultas e informações pessoais.
def cliente_id_atual():
    return session.get("cliente_id")
```

```python
# Garante que o utilizador esteja autenticado antes de aceder a uma rota.
# Caso o utilizador não esteja logado, é redirecionado para a página de login.
# Retorna None quando o acesso é permitido, permitindo que a rota continue.
# Este padrão evita duplicação de código de verificação em várias rotas.
def exigir_login():
    if not esta_logado():
        return redirect(url_for("login"))
    return None


# Garante que apenas utilizadores com role "admin" acedam à rota.
# Primeiro verifica se o utilizador está autenticado.
# Em seguida, valida se o papel do utilizador é administrador.
# Caso não tenha permissão, exibe uma mensagem e redireciona para o dashboard.
def exigir_admin():
    if not esta_logado():
        return redirect(url_for("login"))
    if role_atual() != "admin":
        flash("Não tem permissões para executar essa ação.")
        return redirect(url_for("dashboard"))
    return None
```

```python
# Controla o acesso a funcionalidades destinadas a utilizadores internos
# (staff ou administradores).
# Impede que clientes ou utilizadores não autenticados acedam a estas rotas.
# Utiliza uma lista de roles permitidos para facilitar manutenção futura.
def exigir_staff_ou_admin():
    if not esta_logado():
        return redirect(url_for("login"))
    if role_atual() not in ["admin", "staff"]:
        flash("Não tem permissões para executar essa ação.")
        return redirect(url_for("dashboard"))
    return None


# Garante que apenas clientes autenticados acedam à área do cliente.
# Caso o utilizador não esteja logado, redireciona para o login de clientes.
# Caso esteja logado com outro papel (admin ou staff),
# bloqueia o acesso e redireciona para o dashboard.
def exigir_cliente():
    if not esta_logado():
        return redirect(url_for("login_cliente"))
    if role_atual() != "cliente":
        flash("Área exclusiva para clientes.")
        return redirect(url_for("dashboard"))
    return None
```

Rota: /login

Responsável pela autenticação de utilizadores internos (admin e staff). Valida credenciais, cria a sessão do utilizador e define o seu papel (role), que será usado para controlar o acesso às funcionalidades do sistema.

```python
#---------------------- AUTENTICAÇÃO ------------------------

# Rota responsável pela autenticação de utilizadores internos do sistema,
# como administradores e staff.
# Suporta os métodos GET (exibir formulário) e POST (processar login).
@app.route("/login", methods=["GET", "POST"])
def login():

    # Obtenção dos dados submetidos pelo formulário de login.
    if request.method == "POST":
        username = request.form["username"].strip()
        password = request.form["password"]

        # Ligação à base de dados
        cnx = ligar_bd()
        cur = cnx.cursor(dictionary=True)
        cur.execute(
            "SELECT id, username, password, role FROM users WHERE username=%s",

            # Esta tupla contém os valores que vão substituir os placeholders (%s)
            # A vírgula é obrigatória para indicar que é uma tupla com um único valor
            (username,)
        )

        # Recupera o primeiro registro encontrado (ou None se não existir)
        user = cur.fetchone()
```

```python
        # Fechamento do cursor e da bd
        cur.close()
        cnx.close()

        # Verifica se o utilizador existe e se a password fornecida
        # corresponde à password armazenada na base de dados.
        if user and user["password"] == password:
            session["user_id"] = user["id"]
            session["username"] = user["username"]
            session["role"] = user["role"]

            flash("Login efetuado com sucesso.")
            return redirect(url_for("dashboard"))

        flash("Login inválido")
        return redirect(url_for("login"))

    # Quando a rota é acedida via GET, renderiza o formulário de login.
    return render_template("login.html")
```

Rota: /login_cliente

Permite a autenticação de clientes da clínica.

Este fluxo é separado do login administrativo, garantindo clareza e isolamento entre utilizadores internos e clientes.

```python
# Rota responsável pela autenticação de clientes da clínica.
# Separada do login interno para manter clareza e controlo de permissões.
@app.route("/login_cliente", methods=["GET", "POST"])
def login_cliente():
    if request.method == "POST":

        # Obtém o email e a password do cliente.
        # O email é utilizado como identificador único do cliente.
        email = request.form["email"].strip()
        password = request.form["password"]

        cnx = ligar_bd()
        cur = cnx.cursor(dictionary=True)

        # Procura o cliente na base de dados com base no email fornecido.
        cur.execute("SELECT id, nome, email, password FROM clientes WHERE email=%s",
                    (email,))

        cliente = cur.fetchone()
        cur.close()
        cnx.close()

        if cliente and cliente["password"] == password:
            session["user_id"] = cliente["id"]
            session["username"] = cliente["nome"]

            # role é definido explicitamente como "cliente"
            session["role"] = "cliente"
            session["cliente_id"] = cliente["id"]
```

```python
            flash("Login efetuado com sucesso.")
            return redirect(url_for("dashboard"))

        # Caso o email ou a password estejam incorretos,
        # o cliente é informado e retorna ao login.
        flash("Cliente não encontrado.")
        return redirect(url_for("login_cliente"))

    return render_template("login_cliente.html")
```

Rota: /logout

Encerra a sessão ativa, removendo todos os dados armazenados na sessão e redirecionando o utilizador para a página inicial pública.

```python
# Rota responsável por encerrar a sessão do utilizador,
# independentemente do tipo (admin, staff ou cliente).
@app.route("/logout")
def logout():

    # Remove todos os dados armazenados na sessão,
    # garantindo que o utilizador fique totalmente deslogado.
    session.clear()
    flash("Sessão terminada.")
    return redirect(url_for("index"))
```

Rota: /dashboard

Rota central do sistema após o login.

Apresenta conteúdos diferentes conforme o papel do utilizador:

- cliente: resumo da conta, animais e consultas
- staff: consultas da semana
- admin: visão geral do sistema e estatísticas globais

```python
# ------------ DASHBOARD (QUALQUER AUTENTICADO) --------------

# Rota principal após autenticação.
# Apresenta um dashboard personalizado de acordo com o papel do utilizador:
# - cliente: informações pessoais, animais e consultas
# - staff: agenda de consultas futuras
# - admin: visão geral do sistema (consultas, staff e estatísticas)
@app.route("/dashboard")
def dashboard():

    # Garante que apenas utilizadores autenticados acedam ao dashboard.
    # Caso contrário, redireciona automaticamente para o login.
    redir = exigir_login()
    if redir:
        return redir

    # Obtenção do papel do utilizador e o nome a ser exibido no dashboard.
    # Estes dados serão usados tanto para lógica interna
    # como para apresentação na interface.
    role = session.get("role")
    username = session.get("username")

    cnx = ligar_bd()
    cur = cnx.cursor(dictionary=True)

    # O dicionário "dados" será utilizado para armazenar
    # informações específicas de cada tipo de utilizador.
    dados = {}
```

Rota principal

```python
# ---------- CLIENTE ----------
if role == "cliente":
    cliente_id = session.get("cliente_id")

    # Conta quantos animais estão associados ao cliente autenticado.
    # Este dado é apresentado como informação resumida no dashboard.
    cur.execute(
        "SELECT COUNT(*) AS total FROM animais WHERE cliente_id=%s",
        (cliente_id,)
    )
    dados["total_animais"] = cur.fetchone()["total"]

    # Recupera as próximas consultas do cliente, ordenadas por data e hora.
    # O LIMIT é usado para evitar excesso de dados no dashboard.
    cur.execute(
        """
        SELECT c.data_hora, a.nome AS animal
        FROM consultas c
        JOIN animais a ON c.animal_id = a.id
        WHERE a.cliente_id = %s
        ORDER BY c.data_hora ASC
        LIMIT 5
        """,
        (cliente_id,)
    )
    dados["consultas"] = cur.fetchall()
```

Dashboard cliente

```python
# ---------- STAFF ----------
elif role == "staff":

    # Obtém as consultas agendadas para os próximos 7 dias.
    # Este intervalo facilita a organização do trabalho do staff.
    cur.execute(
        """
        SELECT c.data_hora, a.nome AS animal, cl.nome AS cliente
        FROM consultas c
        JOIN animais a ON c.animal_id = a.id
        JOIN clientes cl ON a.cliente_id = cl.id
        WHERE c.data_hora >= CURDATE()
        AND c.data_hora < DATE_ADD(CURDATE(), INTERVAL 7 DAY)
        ORDER BY c.data_hora ASC
        """
    )
    # retorna a lista de consultas
    dados["consultas"] = cur.fetchall()
```

Dashboard staff

```python
# ---------- ADMIN ----------
elif role == "admin":

    # Consultas da semana, assim como staff
    cur.execute(
        """
        SELECT c.data_hora, a.nome AS animal, cl.nome AS cliente
        FROM consultas c
        JOIN animais a ON c.animal_id = a.id
        JOIN clientes cl ON a.cliente_id = cl.id
        WHERE c.data_hora >= CURDATE()
        AND c.data_hora < DATE_ADD(CURDATE(), INTERVAL 7 DAY)
        ORDER BY c.data_hora ASC
        """
    )
    dados["consultas"] = cur.fetchall()

    # Recupera a lista de utilizadores com role "staff",
    # permitindo ao administrador ter controlo sobre a equipa.
    cur.execute(
        "SELECT username FROM users WHERE role='staff' ORDER BY username"
    )
    dados["staffs"] = cur.fetchall()

    # Estatísticas gerais do sistema,
    # úteis para acompanhamento e tomada de decisão por parte do admin
    cur.execute("SELECT COUNT(*) AS total FROM clientes")
    dados["total_clientes"] = cur.fetchone()["total"]

    cur.execute("SELECT COUNT(*) AS total FROM animais")
    dados["total_animais"] = cur.fetchone()["total"]

    cur.execute("SELECT COUNT(*) AS total FROM consultas")
    dados["total_consultas"] = cur.fetchone()["total"]

cur.close()
cnx.close()

# Envia para o template os dados necessário para renderizar
# o dashboard de forma dinâmica, adaptada ao papel do utilizador autenticado.
return render_template("dashboard.html", role=role, username=username, dados=dados)
```

Dashboard admin

Rota: /users_listar

Lista todos os utilizadores internos do sistema.

Funcionalidade exclusiva para administradores, permitindo controlo centralizado dos acessos.

```
# ---------------- USERS (SOMENTE ADMIN) ---------------------

# Rota responsável por listar todos os utilizadores internos do sistema.
# Apenas administradores têm permissão para aceder a esta funcionalidade.
# Apresenta informações como username, role e data de criação.
@app.route("/users_listar")
def users_listar():
    redir = exigir_admin()
    if redir:
        return redir

    cnx = ligar_bd()
    cur = cnx.cursor(dictionary=True)

    # Recupera todos os utilizadores registados no sistema.
    # A ordenação decrescente permite visualizar os mais recentes primeiro.
    # O resultado é retornado como uma lista de dicionários.
    cur.execute(
        "SELECT id, username, password, role, created_at FROM users ORDER By id DESC"
        )
    lista_users = cur.fetchall()

    cur.close()
    cnx.close()

    # Envia a lista de utilizadores para o template, onde será apresentada em formato de tabela.
    return render_template("users_listar.html", users=lista_users)
```

Rota: /users/novo

Permite ao administrador criar novos utilizadores internos, definindo username, password e papel (admin ou staff).

```python
# Permite ao administrador criar novos utilizadores internos.
# O formulário define username, password e role.
@app.route("/users/novo", methods=["GET", "POST"])
def users_novo():
    redir = exigir_admin()
    if redir:
        return redir

    if request.method == "POST":
        username = request.form["username"].strip()
        password = request.form["password"].strip()
        role = request.form["role"].strip()

        cnx = ligar_bd()
        cur = cnx.cursor()

        cur.execute(
        "SELECT id FROM users WHERE username=%s",
        (username,)
    )
        # Verifica se já existe um utilizador com o mesmo username.
        if cur.fetchone():
            flash("Já existe um utilizador com esse username.")
            cur.close()
            cnx.close()
            return redirect(url_for("users_novo"))

        try:
            # Insere o novo utilizador na base de dados.
            cur.execute(
                "INSERT INTO users (username, password, role) "
                "VALUES (%s, %s, %s)",
                (username, password, role)
            )

            # A transação é confirmada com commit().
            cnx.commit()
            flash("User criado com sucesso!")

        # Captura de erro e feedback ao utilizador
        except mysql.connector.Error as err:
            flash(f"Erro ao criar user: {err}")

        finally:
            cur.close()
            cnx.close()

        return redirect(url_for("users_listar"))

    return render_template("login_form.html", titulo="Novo user", user=None)
```

```python
# Permite ao administrador editar os dados de um utilizador existente.
# O ID do utilizador é passado como parâmetro na URL.
@app.route("/users/editar/<int:id>", methods=["GET", "POST"])
def users_editar(id):
    redir = exigir_admin()
    if redir:
        return redir

    cnx = ligar_bd()
    cur = cnx.cursor(dictionary=True)

    if request.method == "POST":
        username = request.form["username"].strip()
        password = request.form["password"].strip()
        role = request.form["role"].strip()

        # Outro cursor, este para alteração de dados
        # O anterior é para leitura de dados
        # Assim evita-se a mistura de responsabilidades distintas no mesmo cursor
        cur2 = cnx.cursor()

        try:
            # Atualiza as informações do utilizador selecionado.
            cur2.execute(
                "UPDATE users SET username=%s, password=%s, role=%s "
                "WHERE id=%s",
                (username, password, role, id)
            )

            cnx.commit()

            # O rowcount é usado para confirmar se a atualização ocorreu.
            # Caso nenhum registo seja afetado, indica que o ID não existe ou não houve alterações.
            if cur2.rowcount == 0:
                flash("Não foi possível atualizar (ID não encontrado).")
            else:
                flash("Atualizações feitas com sucesso!")

        except mysql.connector.Error as err:
            flash(f"Erro ao atualizar user: {err}")

        finally:
            cur2.close()
            cur.close()
            cnx.close()

        return redirect(url_for("users_listar"))
```

Continua

```
    cur.execute(
        "SELECT id, username, password, role FROM users "
        "WHERE id=%s",
        (id,)
    )

    user_row = cur.fetchone()
    cur.close()
    cnx.close()

    # Se o utilizador não for encontrado
    if not user_row:
        flash("User não encontrado.")
        return redirect(url_for("users_listar"))

    return render_template("login_form.html", titulo="Editar user", login=user_row)
```

Rota: /clientes

Apresenta a lista de clientes registados.

Acessível a admin e staff, sendo a base para a gestão administrativa da clínica.

```
# ------------ CLIENTES (ADMIN: CRUD | STAFF: CRU) --------------

# Lista todos os clientes registados no sistema.
# Acesso permitido apenas a utilizadores internos (admin ou staff).
# Apresenta informações básicas para gestão administrativa.
@app.route("/clientes")
def clientes_listar():

    # Garante que apenas staff ou administradores acedam a esta funcionalidade.
    # Clientes não têm acesso à listagem geral de clientes.
    redir = exigir_staff_ou_admin()
    if redir:
        return redir

    cnx = ligar_bd()
    cur = cnx.cursor(dictionary=True)

    # Recupera todos os clientes registados.
    # A ordenação decrescente facilita a visualização dos registos mais recentes.
    cur.execute(
        "SELECT id, nome, telefone, email, morada, password, created_at "
        "FROM clientes ORDER By id DESC"
        )
    lista_clientes = cur.fetchall()

    cur.close()
    cnx.close()

    # Envia a lista de clientes para o template, onde será apresentada em formato de tabela.
    return render_template("clientes.html",clientes=lista_clientes)
```

Rota: /clientes/novo

Permite criar novos clientes no sistema.

Define automaticamente uma password inicial, que pode ser alterada posteriormente pelo próprio cliente.

```
@app.route("/clientes/novo", methods=["GET", "POST"])
def clientes_novo():
    redir = exigir_staff_ou_admin()
    if redir:
        return redir

    if request.method == "POST":

        # Obtém os dados do cliente a partir do formulário.
        # A password inicial é definida por padrão como "1234",
        # podendo ser alterada posteriormente pelo próprio cliente.
        nome = request.form["nome"].strip()
        telefone = request.form["telefone"].strip()
        email = request.form["email"].strip()
        morada = request.form["morada"].strip()
        password = "1234"

        cnx = ligar_bd()
        cur = cnx.cursor()

        cur.execute(
        "SELECT id FROM clientes WHERE email=%s",
        (email,)
    )
        # Verifica se já existe um cliente registado com o mesmo email.
        # Esta validação garante unicidade do cliente.
        if cur.fetchone():
            flash("Já existe um cliente registado com esse email.")
            cur.close()
            cnx.close()
            return redirect(url_for("clientes_novo"))

        try:

            # Insere o novo cliente na base de dados.
            # A operação é confirmada com commit().
            cur.execute(
                "INSERT INTO clientes (nome, telefone, email, morada, password) "
                "VALUES (%s, %s, %s, %s, %s)",
                (nome, telefone, email, morada, password)
            )

            cnx.commit()

            # Informa o utilizador interno que o cliente foi criado
            # e comunica a password inicial definida.
            flash("Cliente criado com sucesso! Password inicial: 1234")

        except mysql.connector.Error as err:
            flash(f"Erro ao criar cliente: {err}")

        finally:
            cur.close()
            cnx.close()

        return redirect(url_for("clientes_listar"))

    return render_template("clientes_form.html", titulo="Novo cliente", cliente=None)
```

Rota: /clientes/editar/<id>

Permite editar os dados administrativos de um cliente existente, como nome, contacto e morada.

```python
# Permite editar os dados de um cliente existente.
# O ID do cliente é passado como parâmetro na URL.
@app.route("/clientes/editar/<int:id>", methods=["GET", "POST"])
def clientes_editar(id):
    redir = exigir_staff_ou_admin()
    if redir:
        return redir

    cnx = ligar_bd()
    cur = cnx.cursor(dictionary=True)

    if request.method == "POST":
        nome = request.form["nome"].strip()
        telefone = request.form["telefone"].strip()
        email = request.form["email"].strip()
        morada = request.form["morada"].strip()

        cur2 = cnx.cursor()

        try:
            # Atualiza as informações do cliente selecionado.
            # Apenas campos administrativos são alterados.
            cur2.execute(
                "UPDATE clientes SET nome=%s, telefone=%s, email=%s, morada=%s "
                "WHERE id=%s",
                (nome, telefone, email, morada, id)
            )

            cnx.commit()

            # Verifica se o cliente existe e se a atualização foi efetuada.
            if cur2.rowcount == 0:
                flash("Não foi possível atualizar (ID não encontrado).")
            else:
                flash("Atualizações feitas com sucesso!")

        # Tratamento de exceção
        except mysql.connector.Error as err:
            flash(f"Erro ao atualizar cliente: {err}")

        finally:
            cur2.close()
            cur.close()
            cnx.close()

        return redirect(url_for("clientes_listar"))

    cur.execute(
        "SELECT id, nome, telefone, email, morada, password FROM clientes "
        "WHERE id=%s",
        (id,)
    )
```

Continua

```
    cliente_row = cur.fetchone()
    cur.close()
    cnx.close()

    if not cliente_row:
        flash("Cliente não encontrado.")
        return redirect(url_for("clientes_listar"))

    return render_template("clientes_form.html", titulo="Editar cliente", cliente=cliente_row)
```

Rota: /animais

Lista todos os animais registados na clínica, associados aos respetivos clientes.

Acessível apenas a utilizadores internos (admin e staff).

```
# ------------- ANIMAIS (ADMIN + STAFF: CRUD)-----------------

# Lista todos os animais registados no sistema.
# Acesso restrito a administradores e staff.
# Cada animal está associado a um cliente.
@app.route("/animais")
def animais_listar():

    # Garante que apenas utilizadores internos possam aceder à gestão de animais.
    redir = exigir_staff_ou_admin()
    if redir:
        return redir

    cnx = ligar_bd()
    cur = cnx.cursor(dictionary=True)

    # Recupera todos os animais registados no sistema.
    # O resultado é utilizado para apresentação em tabela.
    cur.execute(
        "SELECT id, cliente_id, nome, especie, raca, data_nascimento, created_at FROM animais ORDER By id DESC"
        )
    lista_animais = cur.fetchall()

    cur.close()
    cnx.close()

    # Envia a lista de animais para o template responsável pela apresentação dos dados.
    return render_template("animais.html", animais=lista_animais)
```

Rota: /animais/novo

Permite registar um novo animal, associando-o obrigatoriamente a um cliente existente.

```python
# Permite registar um novo animal associado a um cliente.
# Acesso permitido a admin e staff.
@app.route("/animais/novo", methods=["GET", "POST"])
def animais_novo():
    redir = exigir_staff_ou_admin()
    if redir:
        return redir

    cnx = ligar_bd()
    cur = cnx.cursor(dictionary=True)

    # GET
    # Carrega a lista de clientes para popular o campo <select>
    # no formulário de criação de animais.
    cur.execute("SELECT id, nome FROM clientes ORDER BY nome ASC")
    clientes = cur.fetchall()

    if request.method == "POST":

        # Obtém os dados do animal submetidos pelo formulário.
        # O animal fica obrigatoriamente associado a um cliente.
        cliente_id = request.form["cliente_id"]
        nome = request.form["nome"].strip()
        especie = request.form["especie"].strip()
        raca = request.form["raca"].strip()
        data_nascimento = request.form["data_nascimento"]

        cur2 = cnx.cursor()

        try:

            # Insere o novo animal na base de dados.
            # A associação com o cliente é feita através da chave estrangeira (cliente_id).
            cur2.execute(
                "INSERT INTO animais (cliente_id, nome, especie, raca, data_nascimento) "
                "VALUES (%s, %s, %s, %s, %s)",
                (cliente_id, nome, especie, raca, data_nascimento)
            )

            cnx.commit()
            flash("Animal criado com sucesso!")
            return redirect(url_for("animais_listar"))

        except mysql.connector.Error as err:
            flash(f"Erro ao criar animal: {err}")

        finally:
            cur.close()
            cur.close()
            cnx.close()

    cur.close()
    cnx.close()

    return render_template("animais_form.html", titulo="Novo animal", clientes=clientes)
```

Rota: /animais/apagar/<id>

Remove um animal do sistema.

Funcionalidade restrita ao administrador, garantindo controlo sobre operações destrutivas.

```python
# Permite remover um animal do sistema.
# Acesso restrito ao administrador.
# A operação é definitiva.
@app.route("/animais/apagar/<int:id>", methods=["POST"])
def animais_apagar(id):
    redir = exigir_admin()
    if redir:
        return redir

    cnx = ligar_bd()
    cur = cnx.cursor()

    try:

        # Remove o animal da base de dados.
        # Utiliza o ID como critério de exclusão.
        cur.execute("DELETE FROM animais WHERE id=%s", (id,))
        cnx.commit()

        if cur.rowcount == 0:
            flash("Não existe animal com este ID.")
        else:
            flash("Animal apagado com sucesso.")

    except mysql.connector.Error as err:
        flash(f"Erro ao apagar animal: {err}")

    finally:
        cur.close()
        cnx.close()

    return redirect(url_for("animais_listar"))
```

Rota: /consultas

Apresenta todas as consultas veterinárias registadas no sistema.

Serve como base para o planeamento e acompanhamento clínico.

```python
# -------------- CONSULTAS (ADMIN + STAFF: CRUD)-----------------

# Lista todas as consultas registadas no sistema.
# Acesso restrito a administradores e staff.
# Cada consulta está associada a um animal.
@app.route("/consultas")
def consultas_listar():

    # Garante que apenas utilizadores internos possam aceder à gestão de consultas.
    redir = exigir_staff_ou_admin()
    if redir:
        return redir

    cnx = ligar_bd()
    cur = cnx.cursor(dictionary=True)

    # Recupera todas as consultas registadas no sistema.
    # O resultado é apresentado em formato de tabela.
    cur.execute(
        "SELECT id, animal_id, data_hora, motivo, notas, created_at FROM consultas ORDER By id DESC"
        )
    lista_consultas = cur.fetchall()

    cur.close()
    cnx.close()

    # Envia a lista de consultas para o template responsável pela apresentação das informações.
    return render_template("consultas.html", consultas=lista_consultas)
```

Rota: /consultas/novo

Permite criar uma nova consulta, associando-a a um animal e definindo data, motivo e notas.

```python
# Permite registar uma nova consulta veterinária.
# Acesso permitido a admin e staff.
@app.route("/consultas/novo", methods=["GET", "POST"])
def consultas_novo():
    redir = exigir_staff_ou_admin()
    if redir:
        return redir

    cnx = ligar_bd()
    cur = cnx.cursor(dictionary=True)

    # Carrega a lista de animais para o campo <select>
    # no formulário de criação de consultas.
    cur.execute("SELECT id, nome FROM animais ORDER BY nome ASC")
    animais = cur.fetchall()

    if request.method == "POST":

        # Obtém os dados da consulta submetidos pelo formulário.
        # A consulta fica associada a um animal específico.
        animal_id = request.form["animal_id"].strip()
        data_hora = request.form["data_hora"]
        motivo = request.form["motivo"].strip()
        notas = request.form["notas"].strip()

        cur2 = cnx.cursor()

        try:

            # Insere a nova consulta na base de dados.
            # A data e hora são armazenadas no formato adequado.
            cur2.execute(
                "INSERT INTO consultas (animal_id, data_hora, motivo, notas)" \
                "VALUES (%s, %s, %s, %s)",
                (animal_id, data_hora, motivo, notas)
            )

            cnx.commit()
            flash("Consulta criada com sucesso!")
            return redirect(url_for("consultas_listar"))

        # Tratamento de exceção
        except mysql.connector.Error as err:
            flash(f"Erro ao criar consulta: {err}")

        finally:
            cur2.close()
            cur.close()
            cnx.close()

    cur.close()
    cnx.close()

    return render_template("consultas_form.html", titulo="Nova consulta", consulta=None, animais=animais)
```

Rota: /consultas/apagar/<id>

Remove uma consulta do sistema.

Acesso exclusivo ao administrador, por se tratar de uma ação crítica.

```python
# Permite remover uma consulta do sistema.
# Acesso restrito ao administrador.
# A operação é definitiva.
@app.route("/consultas/apagar/<int:id>", methods=["POST"])
def consultas_apagar(id):
    redir = exigir_admin()
    if redir:
        return redir

    cnx = ligar_bd()
    cur = cnx.cursor()

    try:

        # Remove a consulta da base de dados.
        # Utiliza o ID como critério de exclusão.
        cur.execute("DELETE FROM consultas WHERE id=%s", (id,))
        cnx.commit()

        if cur.rowcount == 0:
            flash("Não existe consulta com este ID.")
        else:
            flash("Consulta apagada com sucesso.")

    except mysql.connector.Error as err:
        flash(f"Erro ao apagar consulta: {err}")

    finally:
        cur.close()
        cnx.close()

    return redirect(url_for("consultas_listar"))
```

Rota: /minha_conta

Permite ao cliente visualizar os seus dados pessoais.

Os dados são filtrados no backend com base no cliente autenticado, garantindo isolamento total da informação.

```python
# ---------------------- ÁREA DOS CLIENTES ------------------------

# Apresenta os dados pessoais do cliente autenticado.
# Acesso exclusivo a utilizadores com role "cliente".
@app.route("/minha_conta")
def minha_conta():

    # Garante que apenas clientes autenticados acedam a esta rota.
    # Utilizadores internos (admin ou staff) não têm acesso.
    redir = exigir_cliente()
    if redir:
        return redir

    cliente_id = session.get("cliente_id")

    cnx = ligar_bd()
    cur = cnx.cursor(dictionary=True)

    # Recupera os dados do cliente com base no ID armazenado na sessão.
    # Isto garante que o cliente só acede aos seus próprios dados.
    cur.execute(
        "SELECT nome, telefone, email, morada, password "
        "FROM clientes WHERE id=%s",
        (cliente_id,)
    )
    cliente = cur.fetchone()

    cur.close()
    cnx.close()

    if not cliente:
        flash("Cliente não encontrado.")
        return redirect(url_for("dashboard"))

    # Envia os dados do cliente para o template
    # para visualização em formato de perfil.
    return render_template("minha_conta.html", cliente=cliente)
```

Rota: /meus_animais

Lista apenas os animais pertencentes ao cliente autenticado, sem possibilidade de acesso a dados de outros clientes.

```python
# Lista todos os animais associados ao cliente autenticado.
# Acesso exclusivo a clientes.
@app.route("/meus_animais")
def meus_animais():
    redir = exigir_cliente()
    if redir:
        return redir

    cliente_id = session.get("cliente_id")

    cnx = ligar_bd()
    cur = cnx.cursor(dictionary=True)

    # Recupera apenas os animais pertencentes ao cliente autenticado.
    # A filtragem por cliente_id garante isolamento dos dados.
    cur.execute(
        "SELECT nome, especie, raca, data_nascimento "
        "FROM animais WHERE cliente_id=%s "
        "ORDER BY nome ASC",
        (cliente_id,)
    )

    animais = cur.fetchall()

    cur.close()
    cnx.close()

    # Envia a lista de animais para o template para apresentação em formato de listagem.
    return render_template("meus_animais.html",animais=animais)
```

Rota: /minhas_consultas

Apresenta todas as consultas associadas aos animais do cliente, ordenadas por data.

```python
# Lista todas as consultas associadas aos animais do cliente autenticado.
# O cliente não consegue visualizar consultas de outros clientes.
@app.route("/minhas_consultas")
def minhas_consultas():
    redir = exigir_cliente()
    if redir:
        return redir

    cliente_id = session.get("cliente_id")

    cnx = ligar_bd()
    cur = cnx.cursor(dictionary=True)

    # Recupera as consultas através de JOIN entre consultas e animais.
    # O filtro por cliente_id garante acesso apenas aos dados do próprio cliente.
    cur.execute(
        "SELECT c.id, a.nome AS animal_nome, c.data_hora, c.motivo, c.notas "
        "FROM consultas c "
        "INNER JOIN animais a ON c.animal_id = a.id "
        "WHERE a.cliente_id = %s "
        "ORDER BY c.data_hora DESC",
        (cliente_id,)
    )

    consultas = cur.fetchall()

    cur.close()
    cnx.close()

    # Envia as consultas do cliente para o template
    # para visualização organizada por data.
    return render_template("minhas_consultas.html", consultas=consultas)
```

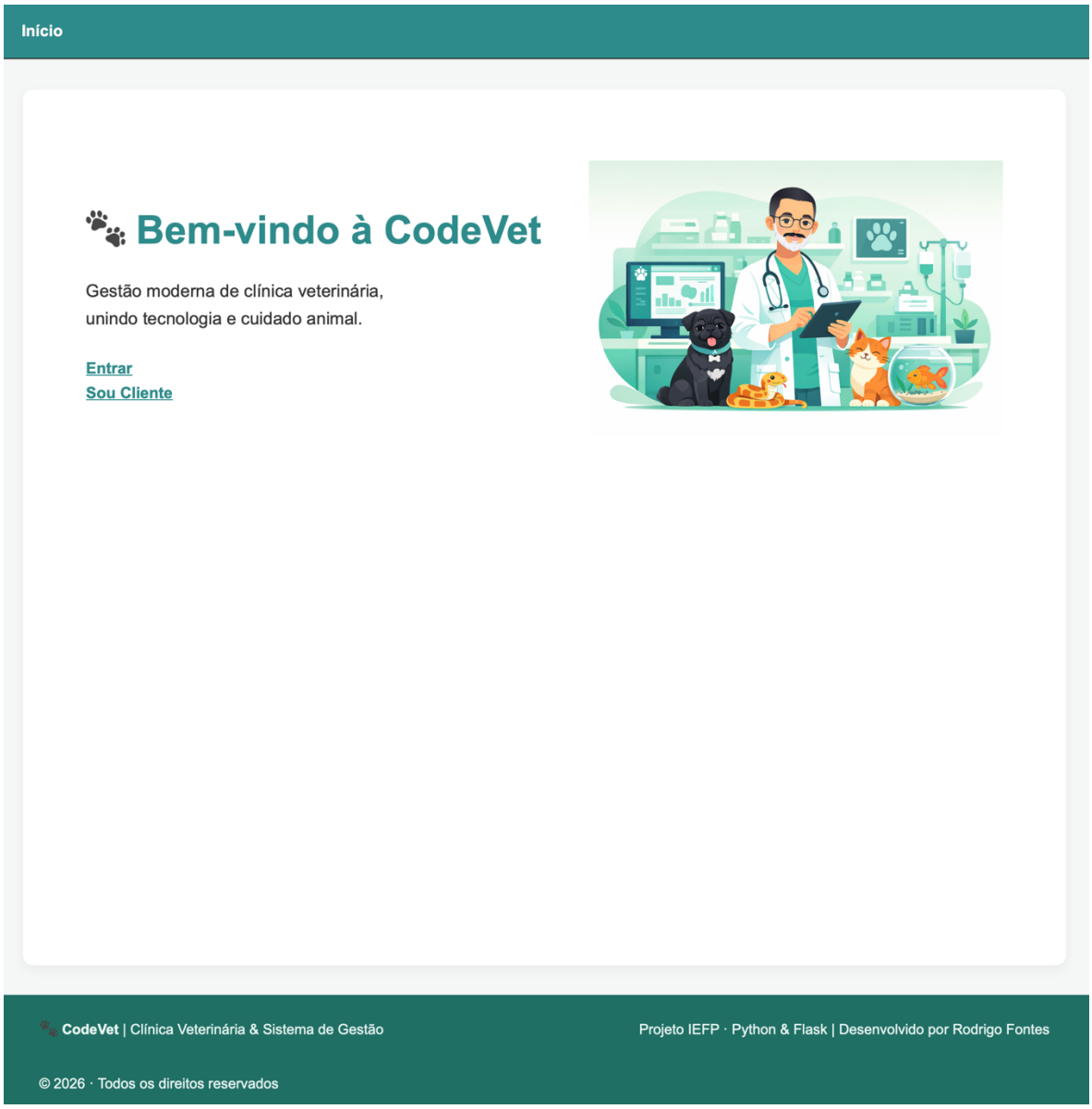
Rota: /mudar_password

Permite ao cliente alterar a sua password, exigindo validação da password atual antes da alteração.

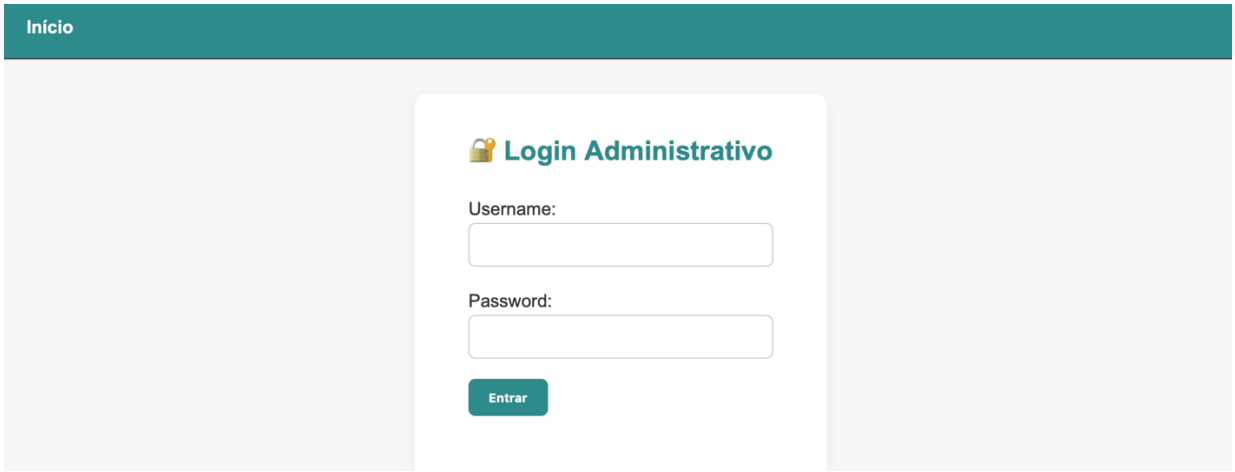
```python
@app.route("/mudar_password", methods=["GET", "POST"])
def mudar_password():
    redir = exigir_cliente()
    if redir:
        return redir

    cliente_id = session.get("cliente_id")

    if request.method == "POST":
        password_atual = request.form["password_atual"]
        nova_password = request.form["nova_password"]

        cnx = ligar_bd()
        cur = cnx.cursor(dictionary=True)

        # Buscar password atual
        cur.execute(
            "SELECT password FROM clientes WHERE id=%s",
            (cliente_id,)
        )
        cliente = cur.fetchone()

        # Verifica se a password atual fornecida corresponde à password armazenada na base de dados.
        # Esta validação impede alterações não autorizadas.
        if not cliente or cliente["password"] != password_atual:
            flash("Password incorreta.")
            cur.close()
            cnx.close()
            return redirect(url_for('mudar_password'))

        # Impede que o cliente defina a mesma password novamente, forçando uma alteração efetiva.
        if password_atual == nova_password:
            flash("A nova password tem de ser diferente da atual.")
            cur.close()
            cnx.close()
            return redirect(url_for('mudar_password'))

        cur2 = cnx.cursor()

        # Atualiza a password do cliente na base de dados.
        # A alteração é persistida com commit().
        cur2.execute(
            "UPDATE clientes SET password=%s WHERE id=%s",
            (nova_password, cliente_id)
            )

        cnx.commit()

        cur.close()
        cur2.close()
        cnx.close()

        # Informa o cliente que a password foi alterada
        # e redireciona para o dashboard.
        flash("Password alterada com sucesso!")
        return redirect(url_for("dashboard"))

    return render_template("mudar_password.html")
```

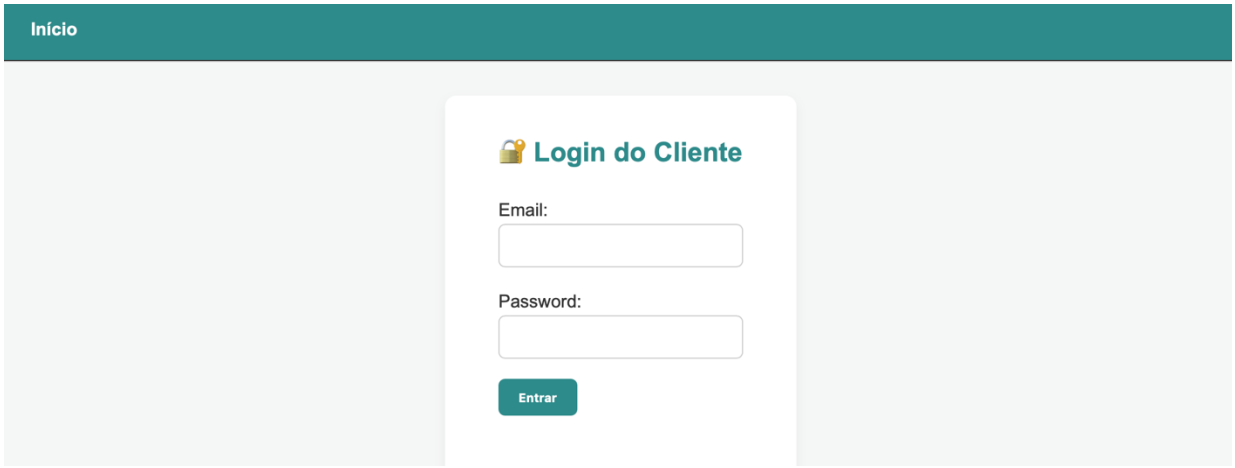
Página Web CodeVet

A página de boas vindas é acessível a qualquer pessoa. Nela estão as opções de entrar como cliente ou funcionário da clínica.

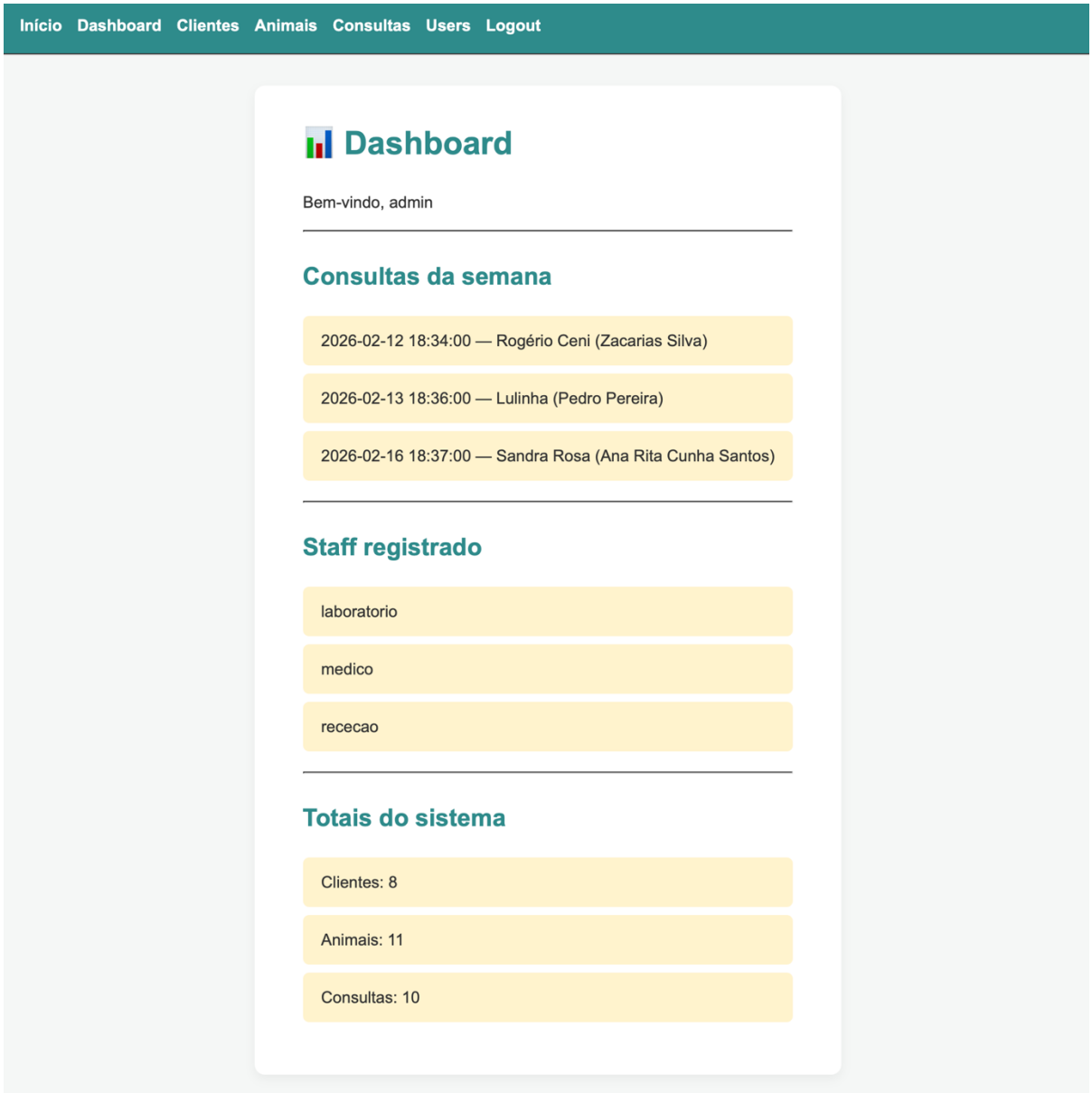


Abaixo, as páginas de login administrativo e de cliente:

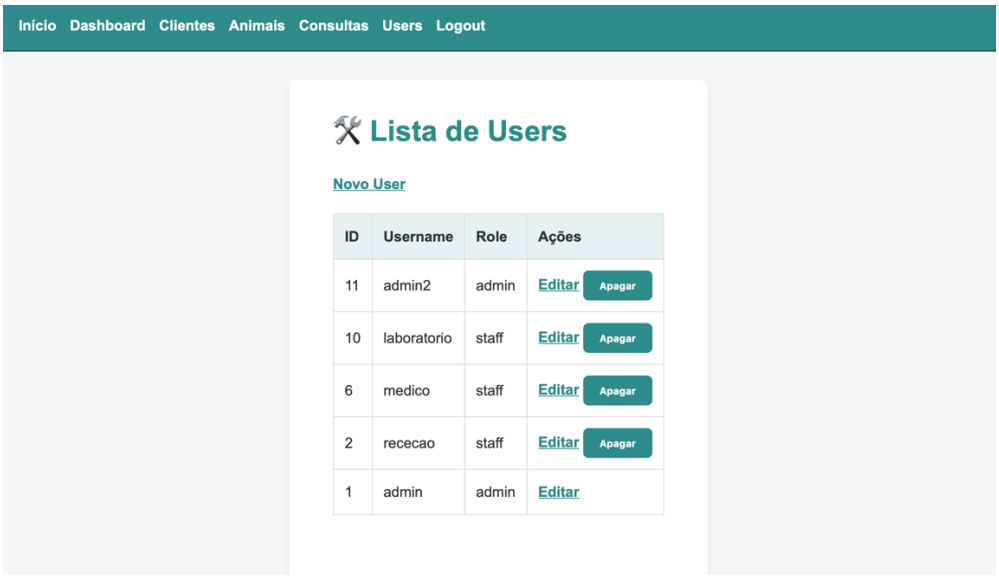




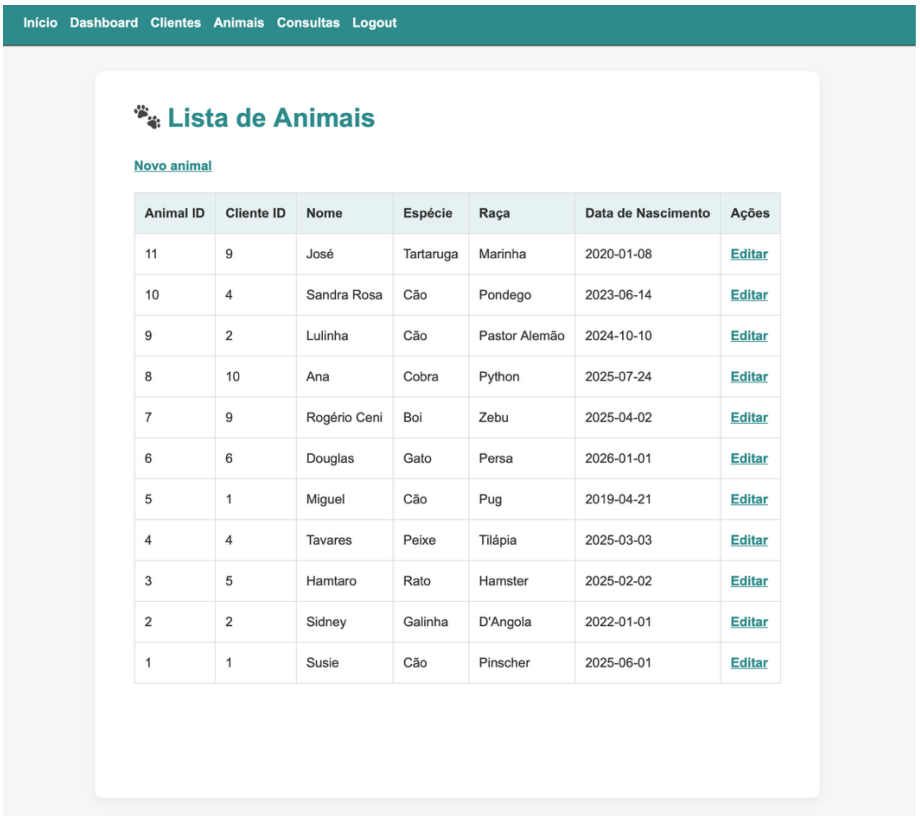
Modelo de dashboard. Neste caso, do utilizador admin:



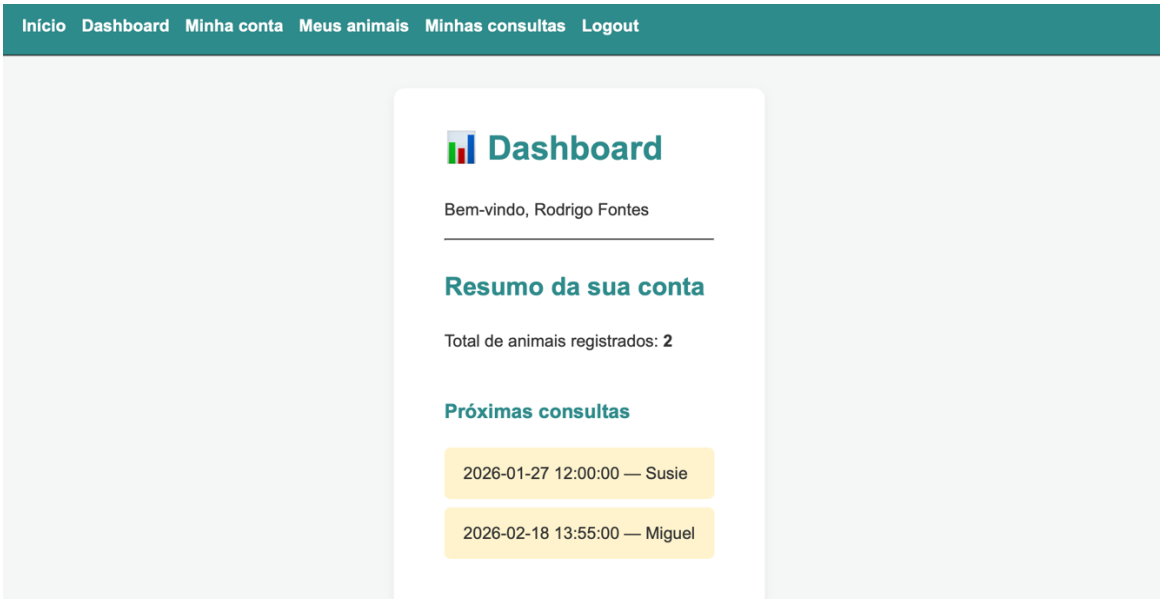
Página da lista de users. Apenas o admin tem acesso a ela.



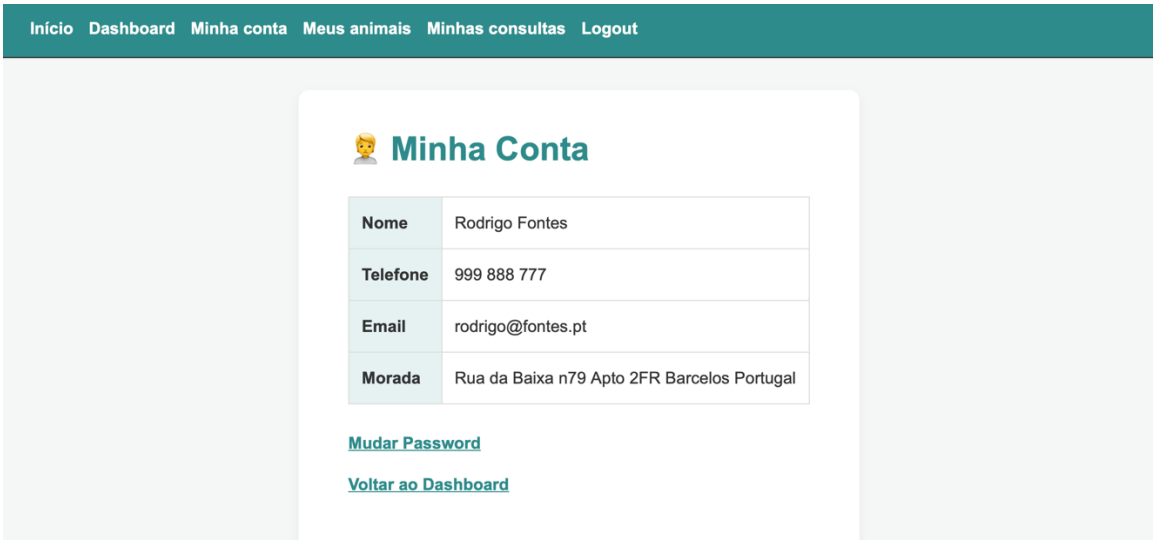
Página de animais listados. Acesso via staff, onde esse tipo de role pode editar mas não apagar.



Dashboard para cliente, onde são mostrados a quantidade de animais registados e as consultas.



Área pessoal do cliente, onde ele pode verificar seus dados da conta e trocar a password.



Considerações Finais

Com a realização do projeto **CodeVet**, foi possível consolidar os conhecimentos adquiridos ao longo da formação, aplicando na prática conceitos fundamentais de programação web com Python e Flask. O desenvolvimento do sistema permitiu trabalhar a criação de rotas, formulários, ligação a bases de dados, utilização de templates HTML e controlo de acessos por tipo de utilizador.

O projeto cumpre os objetivos propostos, apresentando funcionalidades essenciais para a gestão de uma clínica veterinária, como o registo de clientes, animais e consultas, bem como uma área reservada para clientes. A separação entre utilizadores internos e clientes contribui para uma melhor organização e segurança do sistema.

Apesar de se tratar de um projeto académico, a aplicação foi desenvolvida de forma estruturada e funcional, permitindo identificar boas práticas e possíveis melhorias futuras, como o reforço da segurança das passwords e a otimização da interface. De forma geral, o projeto contribuiu para o desenvolvimento de competências técnicas e para uma melhor compreensão do funcionamento de aplicações web completas.